

BCELoss#

<https://pytorch.org/docs/stable/generated/torch.nn.modules.loss.BCELoss.html>

Description:

Creates a criterion that measures the Binary Cross Entropy between the target and the input probabilities: The unreduced (i.e. with reduction set to 'none') loss can be described as: where N is the batch size. If reduction is not 'none' (default 'mean'), then This is used for measuring the error of a reconstruction in for example an auto-encoder. Note that the targets y should be numbers between 0 and 1. Notice that if x_n is either 0 or 1, one of the log terms would be mathematically undefined in the above loss equation. PyTorch chooses to set $\log(0) = -\infty$, since $\lim_{x \rightarrow 0} \log(x) = -\infty$. However, an infinite term in the loss equation is not desirable for several reasons.

Function Signature

```
class torch.nn.modules.loss.BCELoss(weight=None,  
size_average=None, reduce=None, reduction='mean')[source]#
```

Parameters

Shape:

```
forward(input, target)[source]#
```

Return type

Returns:

Tensor

Code Examples

```
>>> m = nn.Sigmoid()  
>>> loss = nn.BCELoss()  
>>> input = torch.randn(3, 2, requires_grad=True)  
>>> target = torch.rand(3, 2, requires_grad=False)  
>>> output = loss(m(input), target)  
>>> output.backward()
```

Detailed Documentation

Documentation

Creates a criterion that measures the Binary Cross Entropy between the target and the input probabilities:

The unreduced (i.e. with reduction set to 'none') loss can be described as:

where N is the batch size. If reduction is not 'none' (default 'mean'), then

This is used for measuring the error of a reconstruction in for example an auto-encoder. Note that the targets y should be numbers between 0 and 1.

Notice that if x is either 0 or 1, one of the log terms would be mathematically undefined in the above loss equation. PyTorch chooses to set $\log(0) = -\infty$, since $\lim_{x \rightarrow 0} \log(x) = -\infty$. However, an infinite term in the loss equation is not desirable for several reasons.

For one, if either $y_n = 0$ or $(1 - y_n) = 0$, then we would be multiplying 0 with infinity. Secondly, if we have an infinite loss value, then we would also have an infinite term in our gradient, since $\lim_{x \rightarrow 0} \frac{d}{dx} \log(x) = \infty$. This would make BCELoss's backward method nonlinear with respect to x , and using it for things like linear regression would not be straight-forward.

Our solution is that BCELoss clamps its log function outputs to be greater than or equal to -100. This way, we can always have a finite loss value and a linear backward method.

weight (Tensor, optional) – a manual rescaling weight given to the loss of each batch element. If given, has to be a Tensor of size $nbatch$.

size_average (bool, optional) – Deprecated (see reduction). By default, the losses are

averaged over each loss element in the batch. Note that for some losses, there are multiple elements per sample. If the field `size_average` is set to `False`, the losses are instead summed for each minibatch. Ignored when `reduce` is `False`. Default: `True`

`reduce` (bool, optional) – Deprecated (see `reduction`). By default, the losses are averaged or summed over observations for each minibatch depending on `size_average`. When `reduce` is `False`, returns a loss per batch element instead and ignores `size_average`. Default: `True`

`reduction` (str, optional) – Specifies the reduction to apply to the output: 'none' | 'mean' | 'sum'. 'none': no reduction will be applied, 'mean': the sum of the output will be divided by the number of elements in the output, 'sum': the output will be summed. Note: `size_average` and `reduce` are in the process of being deprecated, and in the meantime, specifying either of those two args will override `reduction`. Default: 'mean'

Input: $(*)^{(*)} (*)$, where $***$ means any number of dimensions.

Target: $(*)^{(*)} (*)$, same shape as the input.

Output: scalar. If reduction is 'none', then $(*)^{(*)} (*)$, same shape as input.

Runs the forward pass.